A3-PS023-TC-209

MINIMAL TWO-NODE CLOUD HYPERVISOR VM CONTROLLER

Students:
Vansh Vinayak Mohapatra
vvmohapatra@gmail.com
Vijay Siddappa Bandiwaddar
vijaybandiwaddar2008@gmail.com

Mentor:
Prafullata K Auradkar
prafullatak@pes.edu

AIORI Hackathon 3

Problem Description and Background

- Enterprise/Bare-metal/Type 1 hypervisors run directly on the hardware of the host machine without an operating system underneath.
- Examples are Microsoft Hyper-V, Red Hat Enterprise Virtualisation, etc.
- They rely on complex multi-daemon architectures that carry high overhead costs (rent,utilities,administrative pay,etc.) for light-weighted edge deployments..
- They run on dedicated hardware, require a management console and are typically used in data centers.
- They are harder to set up and manage and often depend on external administrative interfaces for management, which can complicate operations.

Problem Description and Background

- Type 2/Hosted hypervisor is a VM manager that is installed as a software application on an existing OS. It does not directly interact with the underlying host machine’s hardware.
- They can be used for non-resource intensive and/or non-business-critical worloads and/or in environments with a small number of servers.
- Examples are VMWare Workstation, Proxmox VE, etc.

Kernel-Based Virtual Machine (KVM)

- KVM is an open source hypervisor integrated into the linux kernel. It leverages Linux’s existing process and memory management capabilities, offering a lean and efficient virtualisation platform.
- **Architecture**: KVM uses a combination of the linux kernel and the Quick Emulator (QEMU) for device emulation. Each VM runs as a separate process, allowing for easy isolation and resource management.
- **Feature**: KVM supports hardware-assisted virtualisation (Intel VT-x and AMD-V), provides strong security through SELinux and sVirt, and integrates seamlessly with Linux management tools.
- **Use Cases**: Preferred in cloud environments, development and testing due to its cost-effectiveness,flexibility and strong integration with Linux-based systems.

Challenges

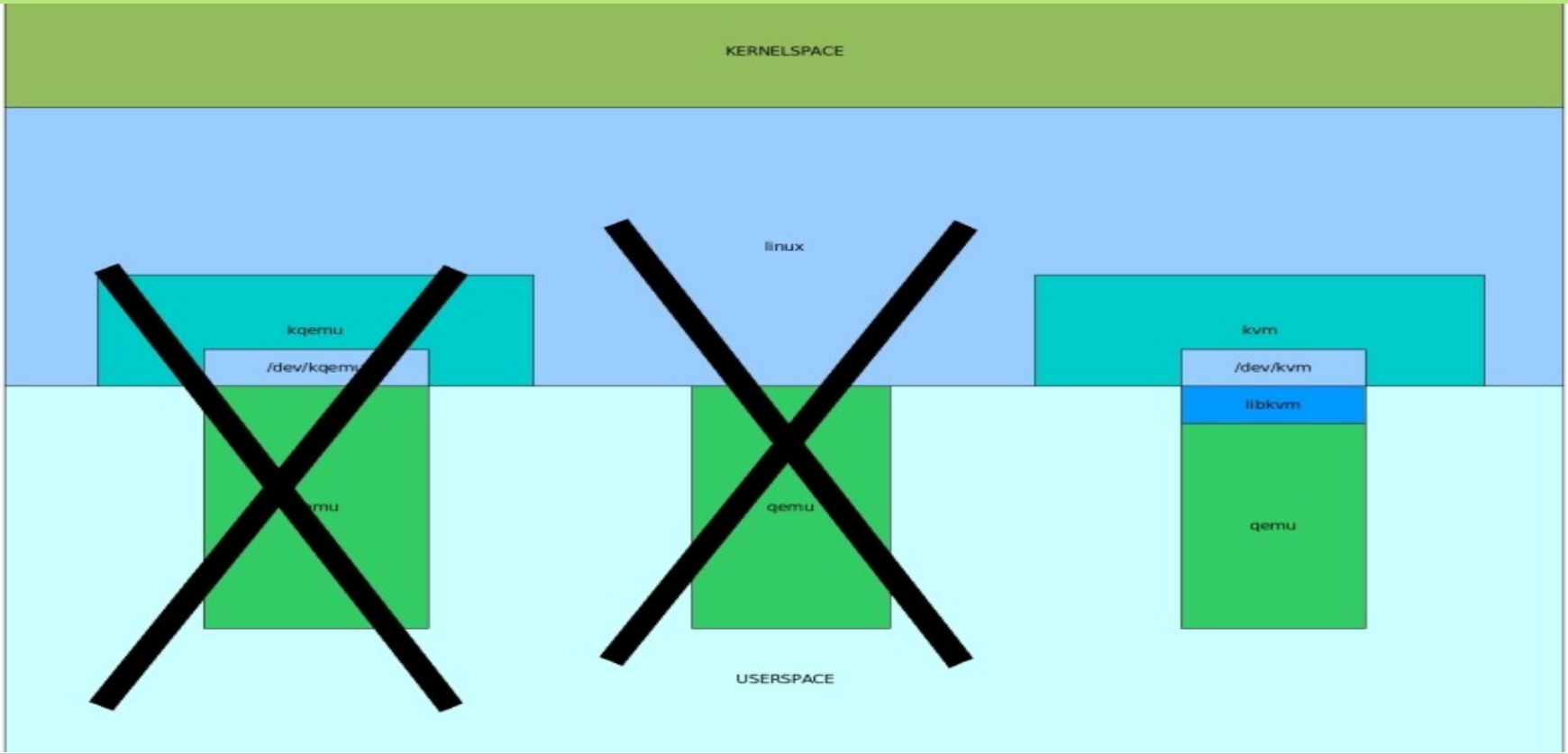
- **Performance overhead**: Managing the balance between virtualisation overhead and native performance.
- **Security**: Constantly evolving threats necessitate advanced security measures and regular updates.
- **Complexity in management**: As the number of VMs grows, managing them effectively becomes challenging, requiring more sophisticated tools and automation.
- **Interoperability**: Ensuring hypervisors can work seamlessly with various hardware, software and cloud environments.

Literature and RFC Review

- KVM and QEMU setup

  QEMU can run as standalone, with kqemu module or with KVM module. In normal mode, only the user code is executed directly and the kernel code is still dynamically translated by QEMU. When KQEMU runs in full virtualisation mode, both guest kernel and user code are executed directly on the host CPU. When run with KVM module, the CPU extensions (svm or vmx) are used.

Literature and RFC Review



Literature and RFC Review

- Memory

  The QEMU/KVM process runs mostly like a normal Linux program. It allocates its memory with normal malloc() or mmap() calls. If a guest is going to have 1GB of physical memory, QEMU/KVM will effectively do a malloc(1<<30), allocating 1GB of host virtual space. However, just like a normal program doing a malloc(), there is no actual physical memory allocated at the time of the malloc().

Literature and RFC Review

- Setting up MMU

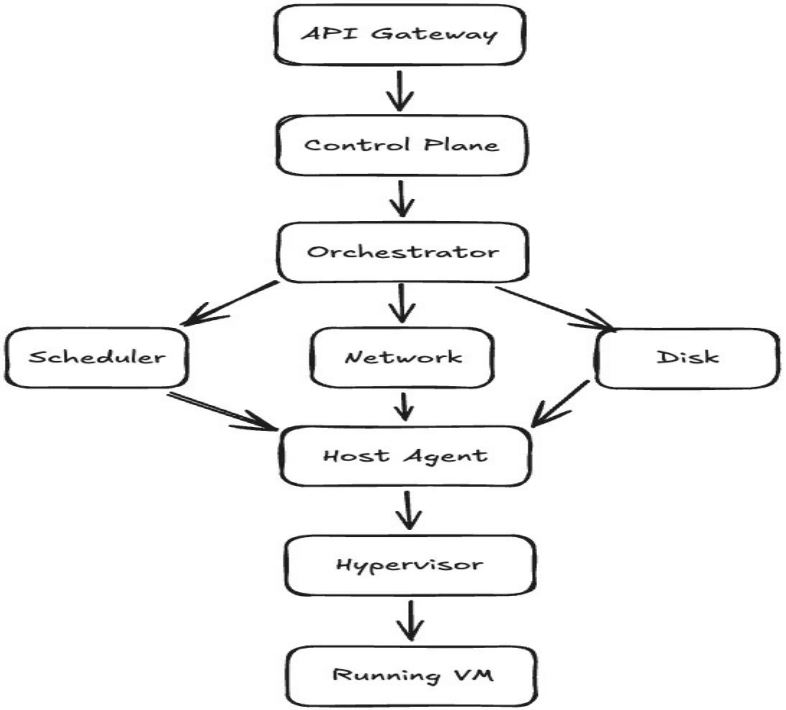
  One of the initialisation steps that KVM does when a VM is started, is setting up the vCPU's Memory Management Unit (MMU) to translate virtual (lineal) addresses into physical ones within the guest's domain.

  For x86 architecture, refer bibliography.

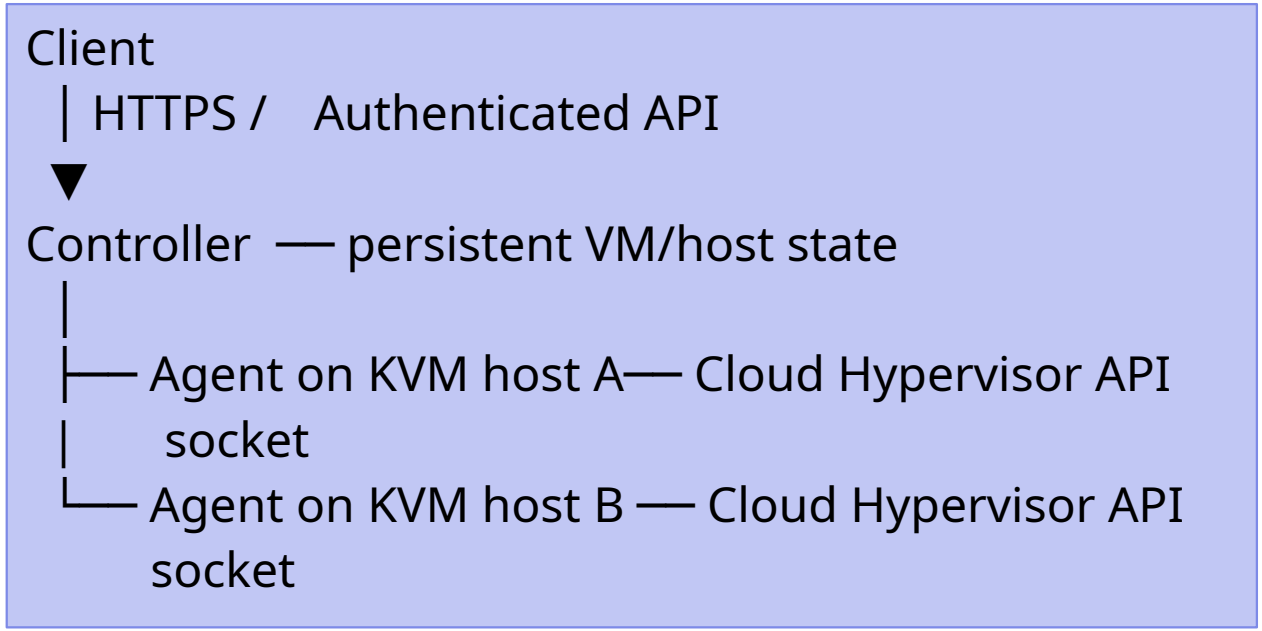
Challenge

- To build a small(minimal) central controller and two worker agents across two KVM-capable hosts. The worker agents manage VM creation, startup, shutdown and listing through the cloud hypervisor management API across the two hosts.

VM Creation and Operation



Solution Design



Definitions

- **Controller**: Accepts create/start/shutdown/list requests, validates them, selects a host for new Vms, records VM-to-host placement, and aggregrates results.
- **Worker Agent**: Runs on its host,launches or supervises cloud hypervisor processes, accesses the local API sockets,reports host health/capcity, and executes lifecycle operations.
- **State store**: Stores VM ID, host ID, requested config. , lifecycle state, and operation status.

Deployment TImeline

- Use 2 /dev/kvm capable hosts with preconfigured networking.

  → Alpine OS is the host on two devices (refer wiki.alpinelinux.org)
- Use 1 prepared VM image.

  → Alpine OS installed in the VM because it is light-weighted and minimal.
- Configure the controller
- Set worker contract

  → Both workers implement the same endpoints and response shapes.

  → The worker should make ‘create/start/stop’ operations idempotent by recording operation ID and VM ID.

  → Its stop handler should call Cloud Hypervisor shutdown, wait for completion, delete the VM from the hypervisor, and clean up its process/socket.

Deployment TImeline

- Authenticate management API

  → Clients to Controller: use a bearer token over https

  → Controller to worker agents: use mTLS (mutual TLS) so that the controller verifies each worker agent’s certificate and each worker agent verifies the controller’s certificate.

  → Add a FastAPI dependency to protect every management route.

- Enforce defined VM resource limit

  → Set a vCPU count, set assigned RAM and a hard-maximum,apply bandwidth limits, set virtual-disk capacity.

Deployment TImeline

- Implement simple placement based on available memory

  → select the host with the most available memory

- Handle API failures

  → Handle API failures by checking the response, catching network errors and returning a useful fallback.

- Lifecycle and two-host placement tests
- Technical report and demonstration.

Bibiliography

- www.linux-kvm.org/page/Documents
- www.jsaer.com/download/vol-8-iss-3-2021/JSAER2021-8-3-278-284.pdf
- https://blog.stgolabs.net/2012/03/kvm-virtual-x86-mmu-setup.html?m=1
- https://wiki.alpinelinux.org/wiki/KVM
- www.awstip.com

Thank You
For Your Attention